

Class 7: Machine Learning 1

Mikaela Joya (PID: A17295169)

Table of contents

Background	1
K-means clustering	1
Principal Component Analysis (PCA)	9
1. Analysis of UK Food Data	9
Data Import	9
Tidy data	10
Exploratory analysis	11
PCA to the rescue	17
Digging deeper (variable loadings)	22
2. PCA of RNA-seq Data	24

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

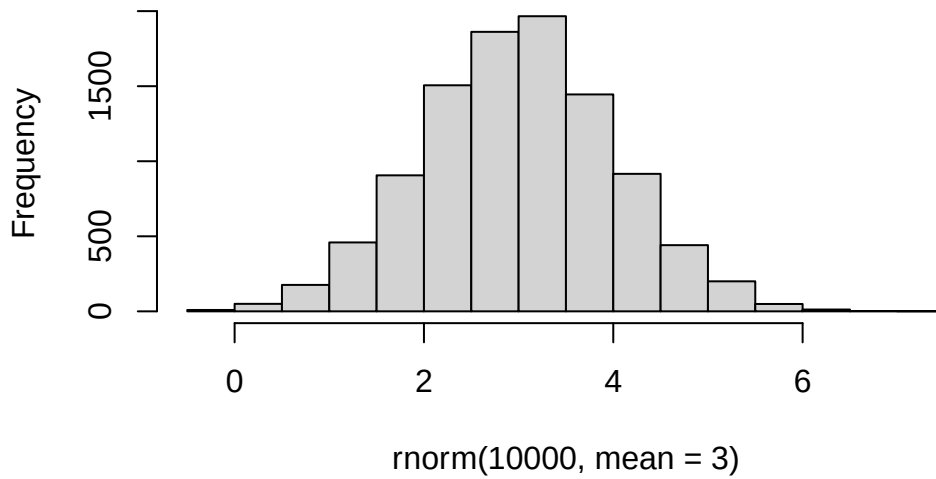
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” dots that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist( rnorm(10000, mean=3) )
```

Histogram of rnorm(10000, mean = 3)



```
x <- c( rnorm(30, -3), rnorm(30, +3) )  
x
```

```
[1] -1.7321150 -1.6066111 -3.6078055 -1.6602528 -2.3964228 -0.7988875  
[7] -3.2802992 -3.5037826 -4.4956046 -1.9711593 -3.4575442 -4.0294190  
[13] -2.8873250 -1.1760792 -1.8687816 -2.9082246 -3.6265869 -2.4572484  
[19] -3.3950329 -2.9302817 -3.7457334 -3.8214255 -4.5565529 -1.3024628  
[25] -2.4760671 -2.1997618 -3.1590198 -2.7357260 -4.2019823 -1.7505514  
[31]  1.1028154  2.8184026  3.1709531  3.7519101  3.5131643  3.3446547  
[37]  2.8151679  4.3243127  2.5158968  3.0859882  4.6943017  2.6350183  
[43]  4.0647046  4.0071384  2.3753072  1.9840826  2.7432210  3.5280582  
[49]  2.1336565  3.9459320  3.0345136  3.5315899  2.6326292  2.0832106  
[55]  3.5783247  4.2364343  3.1929714  2.1438181  3.2337286  2.6581978
```

```
z <- cbind(x, y=rev(x))  
plot(z)
```



```
km <- kmeans(z, centers = 2)
km
```

Cluster means:

Clustering vector:

Within cluster sum of squares by cluster:

3

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
$class
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What ‘component’ of your result object details - cluster size? - cluster assignment/membership? - cluster center?

```
km$cluster
```

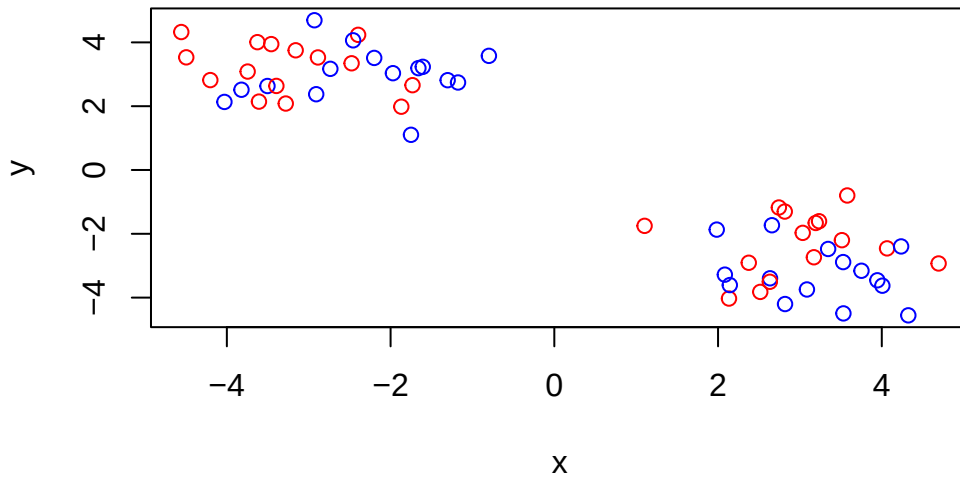
```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
km$centers
```

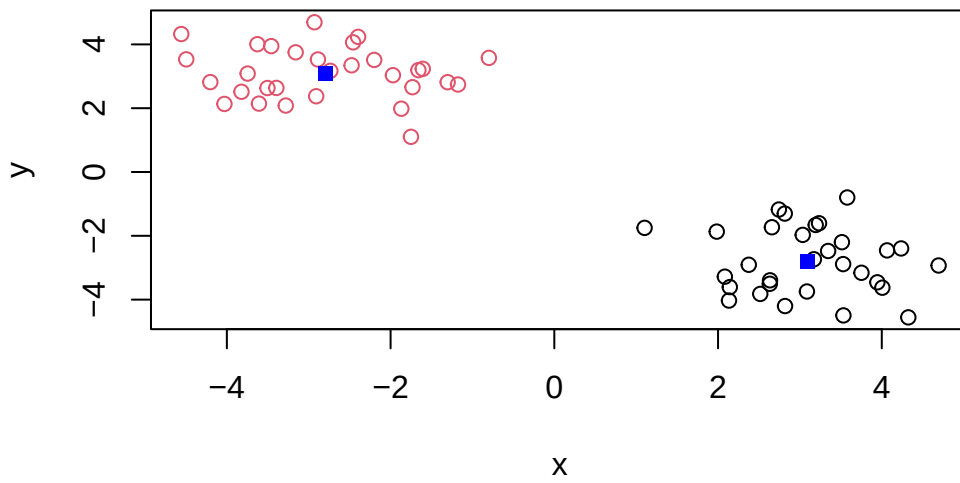
```
      x      y
1  3.096003 -2.791292
2 -2.791292  3.096003
```

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col=c("red","blue") )
```

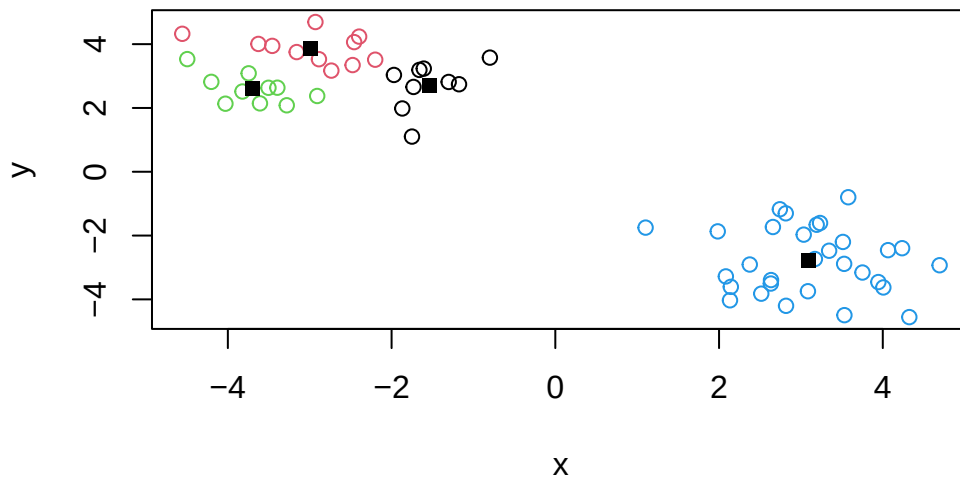


```
plot(z, col=km$cluster)
points(km$centers, col="blue", pch=15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

```
km4 <- kmeans(z, centers = 4)
plot(z, col=km4$cluster)
points(km4$centers, col="black", pch=15)
```

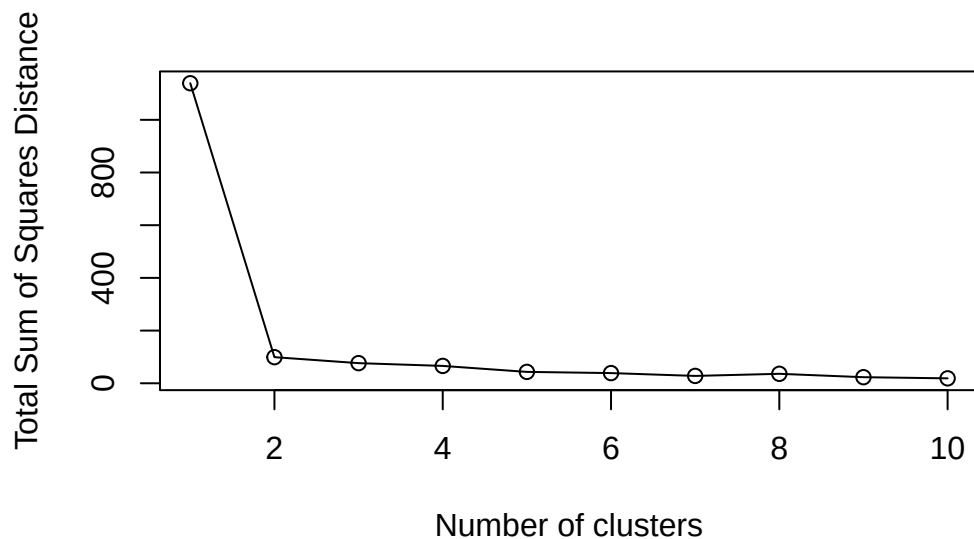


N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best...

```
ans <- NULL
for(i in 1:10) {
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, typ="o",
      xlab="Number of clusters",
      ylab="Total Sum of Squares Distance")
```



Hierarchial Clustering

The main function in “base” R for Hierarchial Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`.

```
d <- dist(z)
hc <- hclust(d)
hc
```

Call:

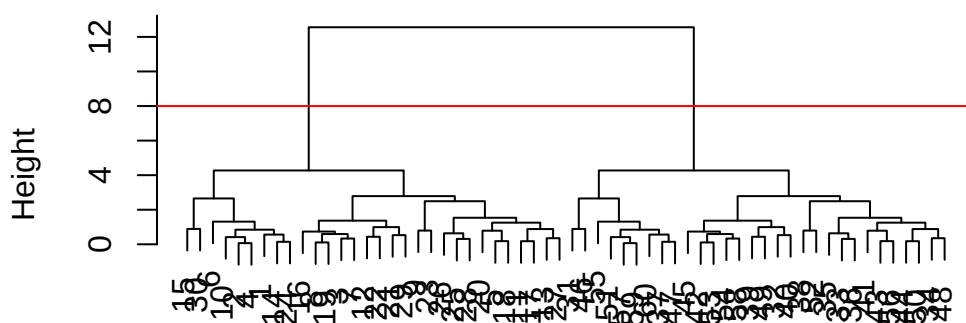
```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col="red")
```

Cluster Dendrogram



d
hclust (*, "complete")

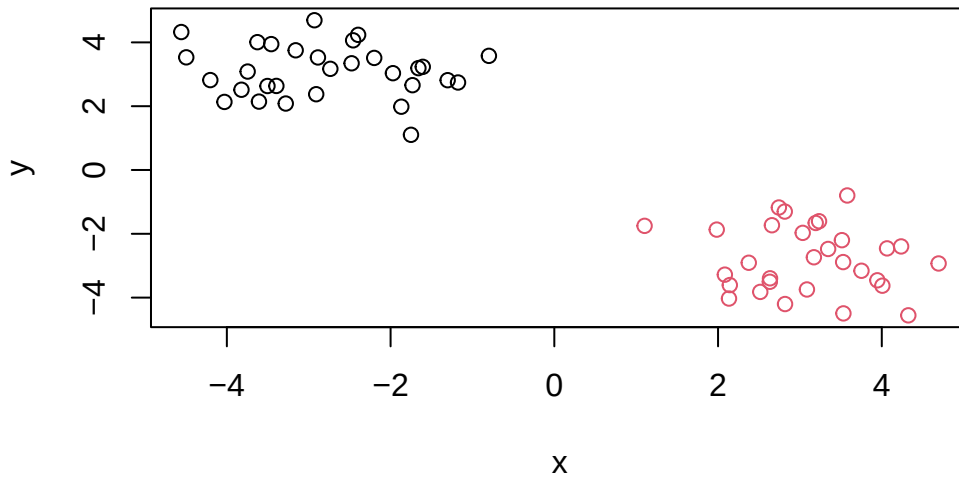
Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Q. Make a plot of `z` with your `hclust` results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k=2)
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

1. Analysis of UK Food Data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function for the import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Tidy data

Fix anything that went wrong with data import.

Q1. How many rows and columns are in your new data frame named `x`? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

Initially, there are 17 rows and 5 columns. But later when fixed, there are 17 rows and 4 columns. To answer this: `dim(x)` was used, but `nrow(x)` and `ncol(x)` could be used too.

Here is the preview of the first 6 rows:

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

It appears that the row-names are incorrectly set as the first column of our `x` data frame (rather than set as proper row-names). Let's fix this!

```
# Note how the minus indexing works
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

This looks much better, now lets check the dimensions again:

```
dim(x)
```

```
[1] 17  4
```

N.B. An alternative approach to setting the correct row-names in this case would be to read the data file again and this time set the `row.names` argument of `read.csv()` to be the first column (i.e. use argument setting `row.names=1`), see below:

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

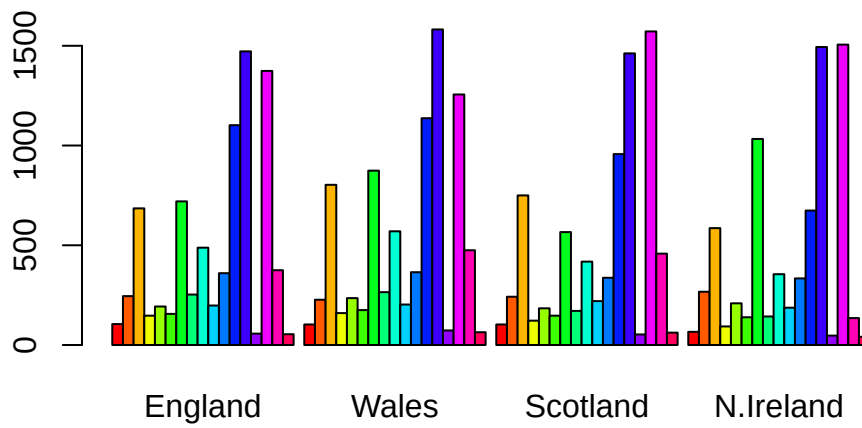
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer using `x <- read.csv(url, row.names=1)` because it fixes the row names when imported avoiding extra steps. This is more robust because if `x <- x[,-1]` is run multiple times, it will keep removing columns and will mess up the data.

Exploratory analysis

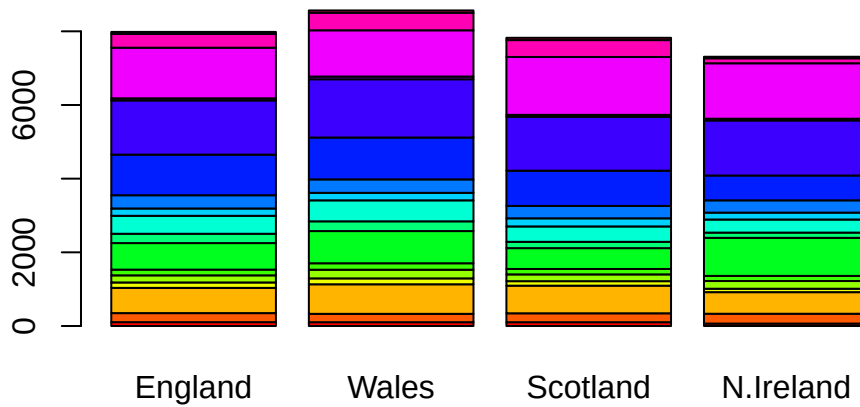
Make some plots to help make sense of obvious trends...

```
# Using base R
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

```
# Using base R
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



Changing the `beside` argument results in the plot changing from a grouped barplot to a stacked barplot. When `beside = TRUE`, the food categories are shown next to each other for each country. While, `beside = FALSE` makes the food categories stacked combining each country.

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

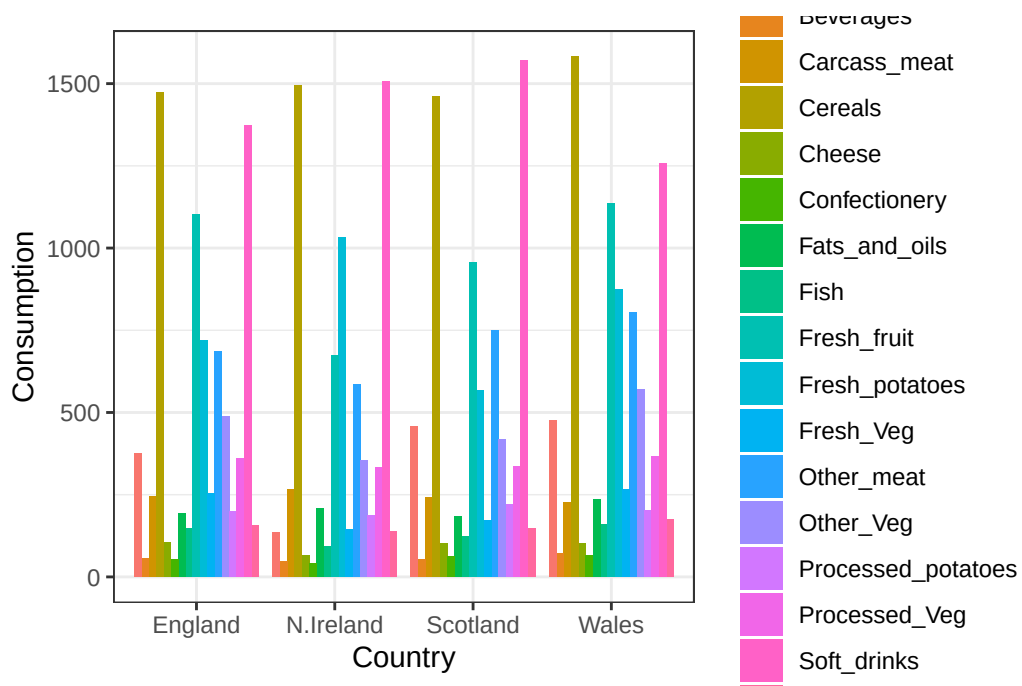
dim(x_long)
```

```
[1] 68  3
```

```
# Create grouped bar plot
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
```

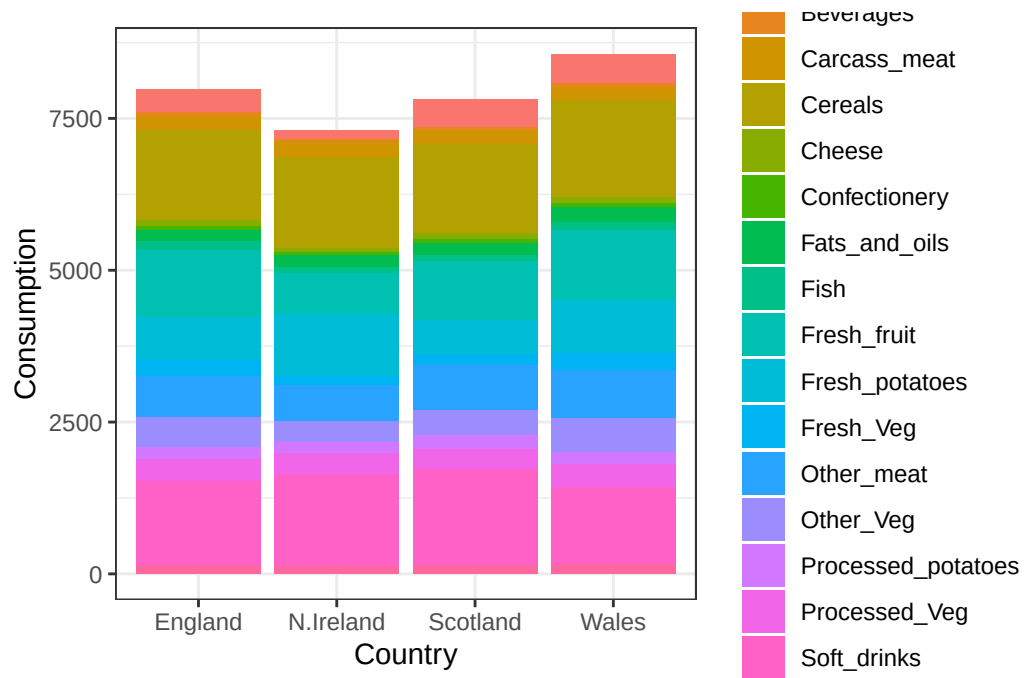
```
geom_col(position = "dodge") +  
theme_bw()
```



Q4. Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

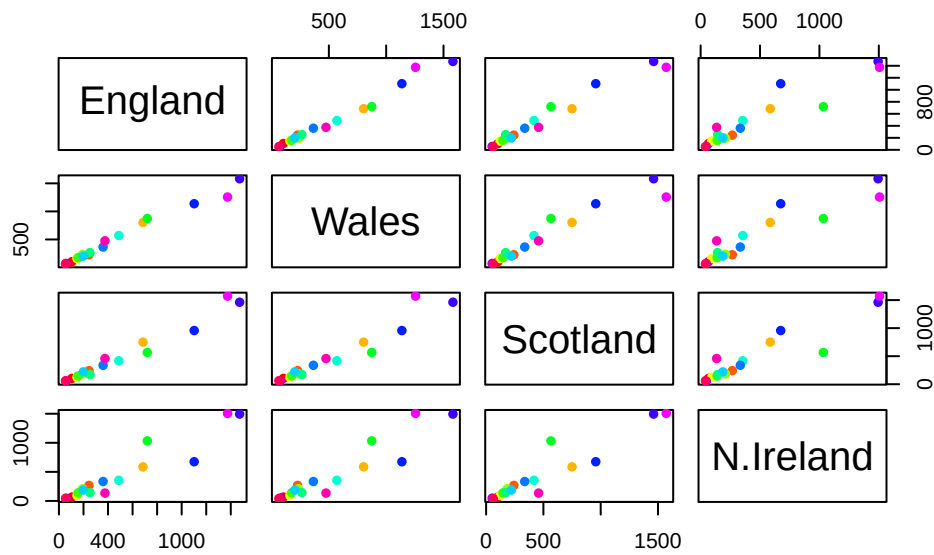
Changing the position argument in `geom_col()` from "dodge" to "stack" results in a stacked barplot figure.

```
library(ggplot2)  
  
ggplot(x_long) +  
  aes(x = Country, y = Consumption, fill = Food) +  
  geom_col(position = "stack") +  
  theme_bw()
```



Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

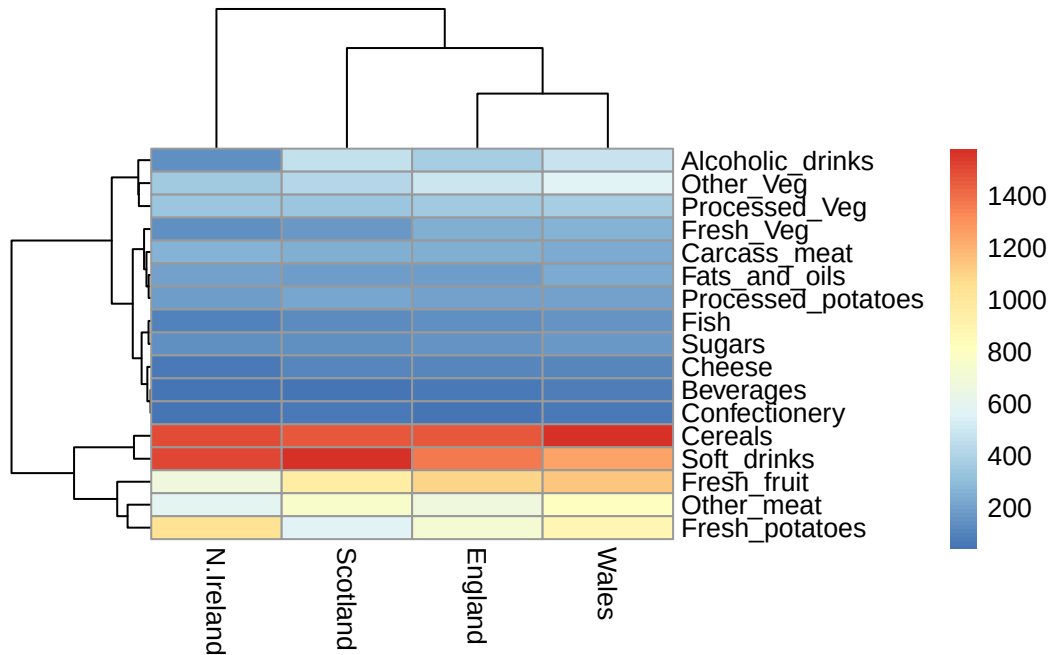
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The `pairs()` function creates scatter plots that compare each country to every other country. Each point is a food type, and if a point is close to the diagonal it means that the food was consumed in similar amounts in the countries compared.

Another useful visualization approach that is frequently used in bioinformatics is a heatmap.

```
library(pheatmap)
pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

The countries that cluster together more closely (having similar food consumption patterns) in the heat map are England, Wales, and Scotland. Northern Island is more separated from the other countries (has different food consumption patterns). This is especially true because when looking at the heatmap, Northern Ireland is different in foods such as fresh potatoes, fresh fruit, and alcoholic drinks.

Key-point: Even relatively small datasets can prove challenging to interpret. Given that it is quite difficult to make sense of even this relatively small data set. Hopefully, we can clearly see that a powerful analytical method is absolutely necessary if we wish to observe trends and patterns in larger datasets.

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object

```
pca <- prcomp ( t(x) )
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main results figures:

```
attributes(pca)
```

```
$names
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

```
$class
[1] "prcomp"
```

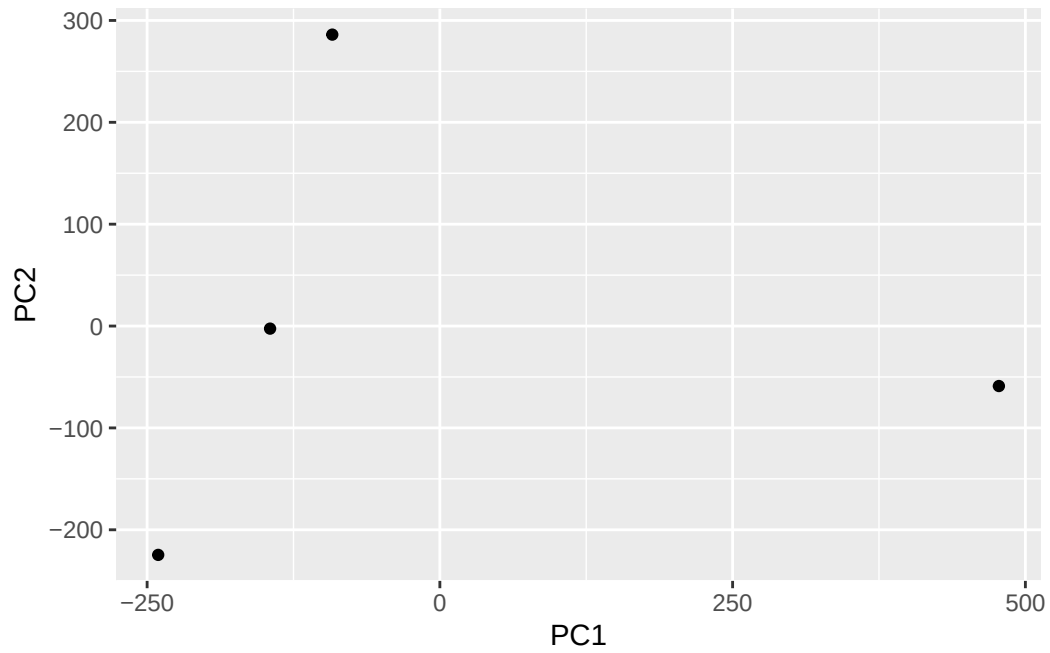
The main result figure from this analysis is called a “**PC score plot**” (a.k.a an “ordination plot,” “PC plot”, or simply “PC1 vs PC2 plot”).

This plot shows how samples (in this case countries) relate to each other along our new PC axis.

This is our new “reduced-dimensional space”. In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimensional data-set.

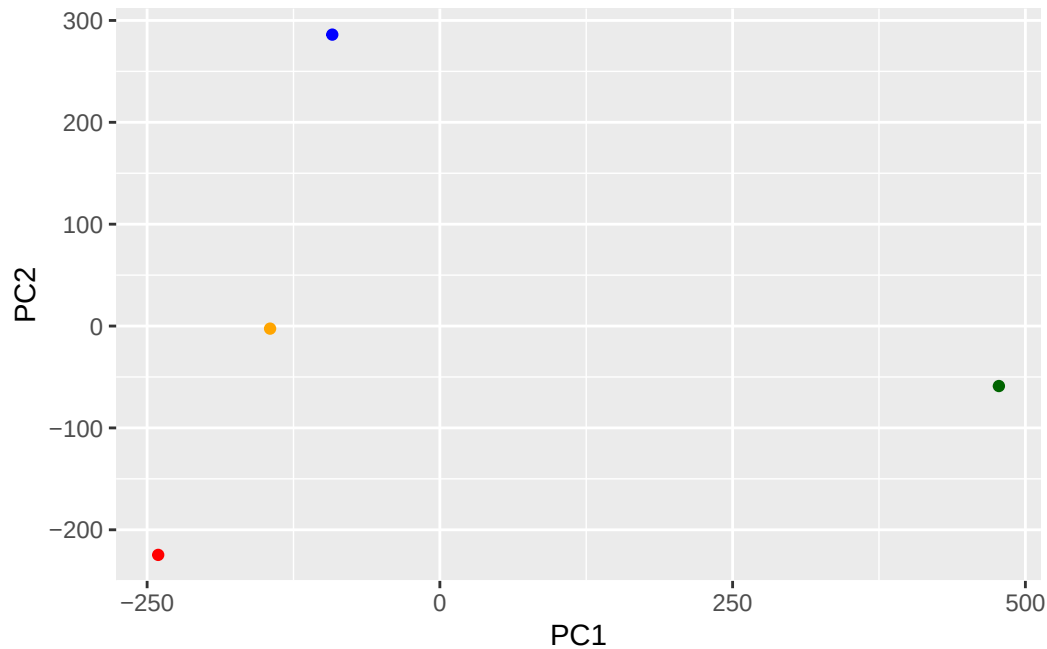
```
library(ggplot2)

ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point()
```



```
mycols <- c("orange", "red", "blue", "darkgreen")

ggplot(pca$x) +
  aes(PC1, PC2, label=row.names(pca$x)) +
  geom_point(col=mycols)
```



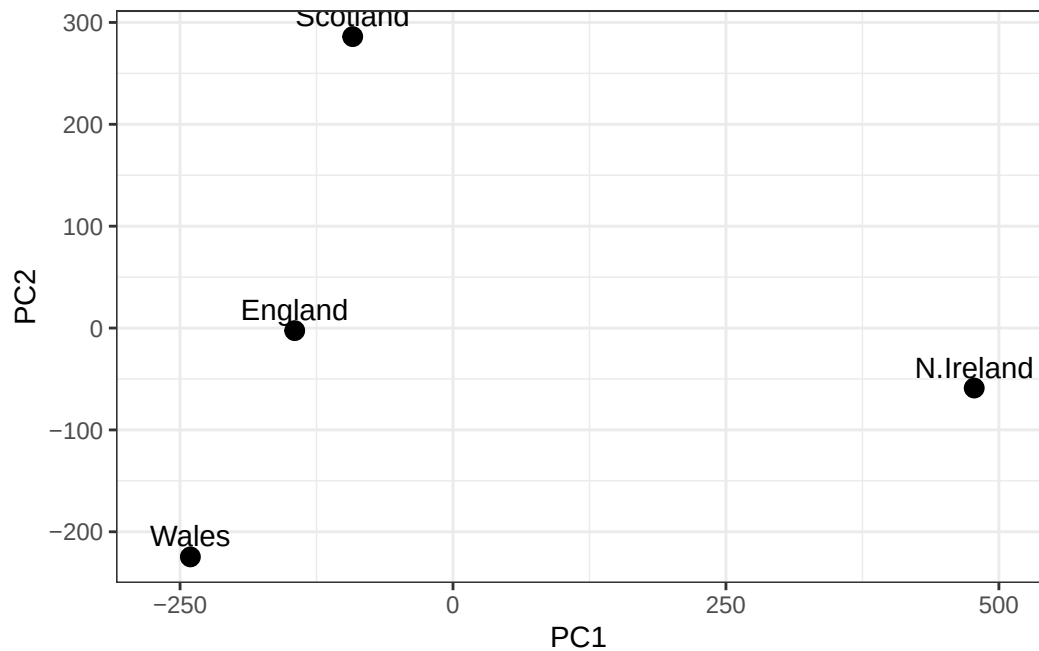
```
geom_text(size=3, vjust=2, col=mycols)
```

geom_text: na.rm = FALSE, parse = FALSE, check_overlap = FALSE, size.unit = mm
 stat_identity: na.rm = FALSE
 position_nudge

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

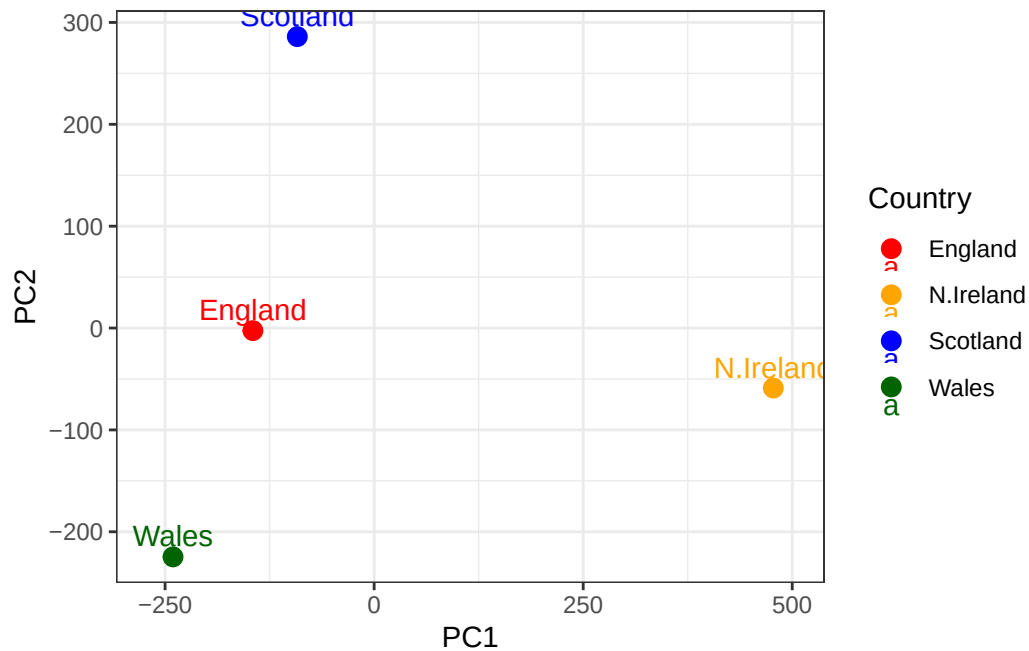
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
country_colors <- c("England" = "red", "Wales" = "darkgreen",
  "Scotland" = "blue", "N.Ireland" = "orange")

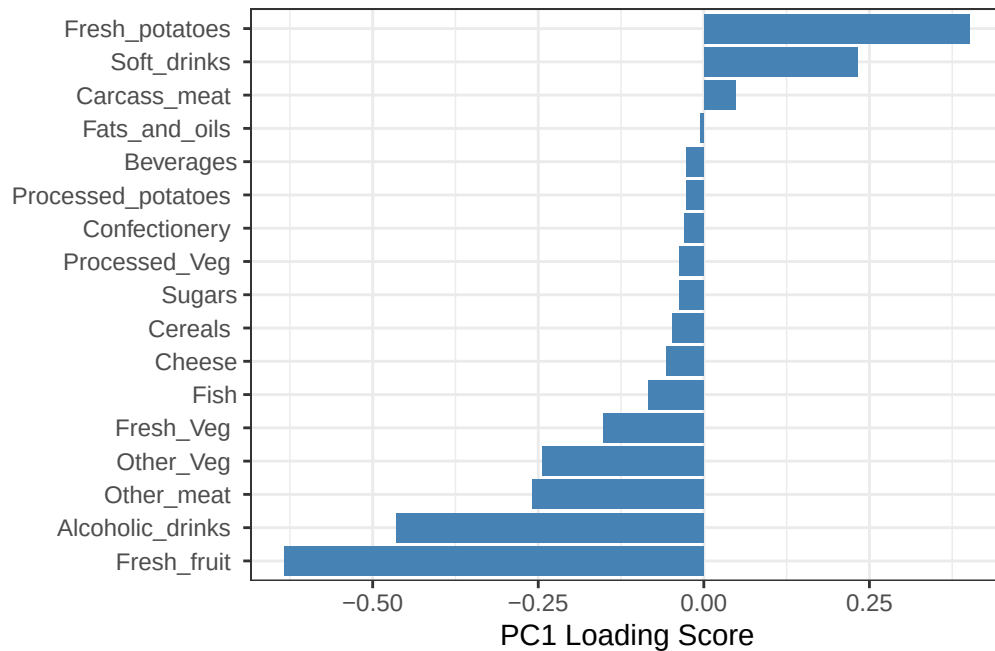
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, color = Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  scale_color_manual(values = country_colors) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Digging deeper (variable loadings)

We can also consider the influence of each of the original variables upon the principal components (typically known as loading scores). This information can be obtained from the `prcomp()` returned `$rotation` component. It can also be summarized with a call to `biplot()`, see below:

```
ggplot(pca$rotation) +
  aes(PC1,
       reorder(row.names(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

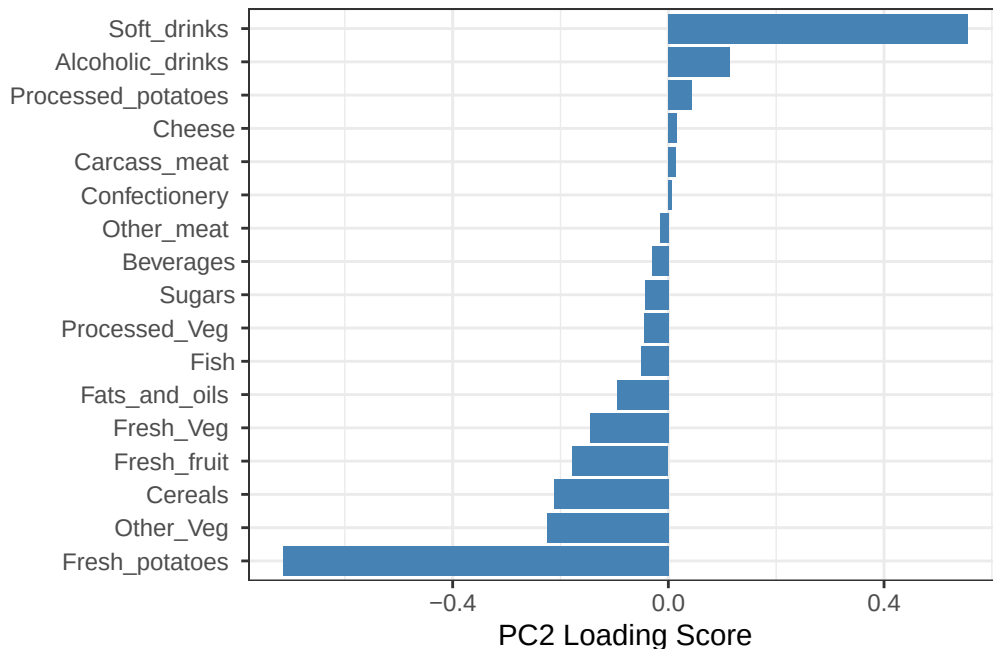


Here we see observations (foods) with the largest positive loading scores that effectively “push” N. Ireland to right positive side of the plot (including Fresh_potatoes and Soft_drinks).

We can also see the observations/foods with high negative scores that push the other countries to the left side of the plot (including Fresh_fruit and Alcoholic_drinks).

Q9. Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



For PC2, the food groups that are prominently featured are the soft drinks and fresh potatoes/fresh fruit (it depends which side of PCS loaded). Furthermore, PC2 mainly tells us about the foods that separate the countries along the second largest variation (after PC1), explaining the smaller differences between countries that PC1 doesn't.

Key-point: Key-Point: PCA has the awesome ability to effectively reduce the dimensionality of our data set down from 17 to 2, allowing us to assert (using our figures above) that countries England, Wales and Scotland are 'similar' with Northern Ireland being different in some way. Furthermore, digging deeper into the loadings we were able to associate certain food types with each cluster of countries. This is super useful!

2. PCA of RNA-seq Data

RNA-seq results often contain a PCA (or related MDS plot). Usually we use these graphs to verify that the control samples cluster together.

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

```
wt1 wt2 wt3 wt4 wt5 ko1 ko2 ko3 ko4 ko5
```

gene1	439	458	408	429	420	90	88	86	90	93
gene2	219	200	204	210	187	427	423	434	433	426
gene3	1006	989	1030	1017	973	252	237	238	226	210
gene4	783	792	829	856	760	849	856	835	885	894
gene5	181	249	204	244	225	277	305	272	270	279
gene6	460	502	491	491	493	612	594	577	618	638

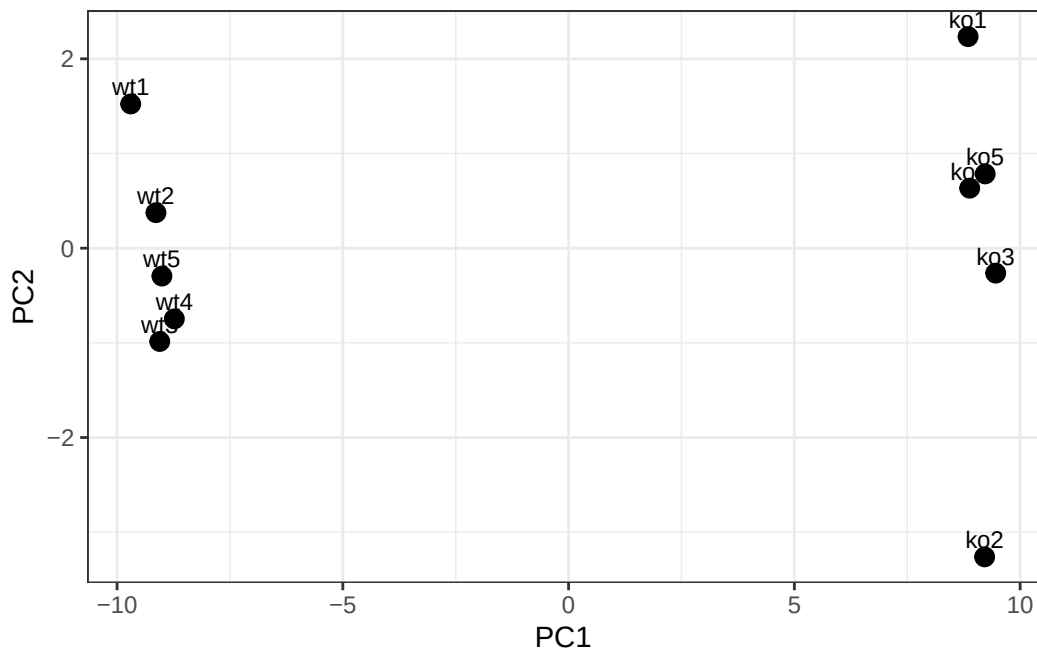
```
dim(rna.data)
```

```
[1] 100  10
```

```
## Again we have to take the transpose of our data
pca <- prcomp(t(rna.data), scale=TRUE)

# Create data frame for plotting
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)

## Plot with ggplot
ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



```
summary(pca)
```

Importance of components:

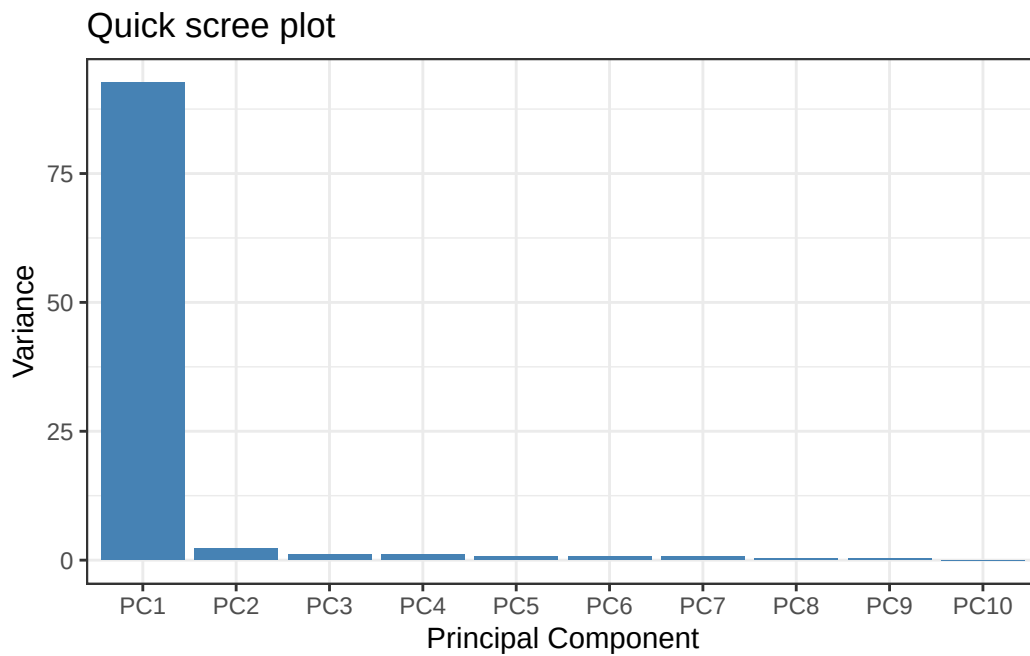
	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	9.6237	1.5198	1.05787	1.05203	0.88062	0.82545	0.80111
Proportion of Variance	0.9262	0.0231	0.01119	0.01107	0.00775	0.00681	0.00642
Cumulative Proportion	0.9262	0.9493	0.96045	0.97152	0.97928	0.98609	0.99251

	PC8	PC9	PC10
Standard deviation	0.62065	0.60342	3.3e-15
Proportion of Variance	0.00385	0.00364	0.0e+00
Cumulative Proportion	0.99636	1.00000	1.0e+00

```
# Calculate variance explained
pca.var <- pca$sdev^2
pca.var.per <- round(pca.var/sum(pca.var)*100, 1)

# Create scree plot data
scree_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  Variance = pca.var[1:10]
)
```

```
ggplot(scree_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  ggtitle("Quick scree plot") +
  xlab("Principal Component") +
  ylab("Variance") +
  theme_bw()
```



Let's make another version showing percent variation:

```
## Percent variance is often more informative to look at
pca.var.per
```

```
[1] 92.6  2.3  1.1  1.1  0.8  0.7  0.6  0.4  0.4  0.0
```

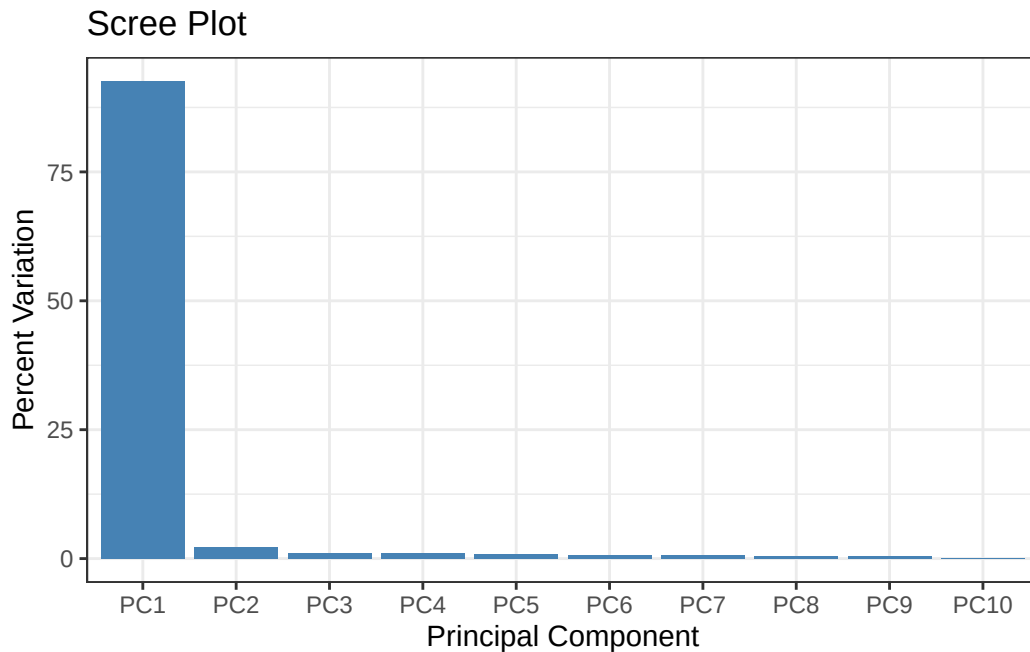
```
# Create percent variance scree plot
scree_pct_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  PercentVariation = pca.var.per[1:10]
)

ggplot(scree_pct_df) +
```

```

aes(x = PC, y = PercentVariation) +
geom_col(fill = "steelblue") +
ggtitle("Scree Plot") +
xlab("Principal Component") +
ylab("Percent Variation") +
theme_bw()

```



Q10. How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see above)?

The RNA-seq dataset has 100 genes and 10 samples. Since PC1 explains 92.6% of the variance, 1 PC is already very useful. Therefore using PC1 and PC2 together gives a better overview because they explain about 94.9% of the variance in total.

Now lets make our main PCA plot a bit more attractive and useful...

```

## A vector of colors for wt and ko samples
colvec <- colnames(rna.data)
colvec[grep("wt", colvec)] <- "red"
colvec[grep("ko", colvec)] <- "blue"

# Add condition to data frame
df$condition <- substr(df$Sample, 1, 2)

```

```
df$color <- colvec

ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```

